Image Segmentation
A Beginner-Friendly Learning Guide

Everything you need to understand — from pixels to a trained model — **before writing a single line of project code.**

This guide assumes you already know basic Python and basic machine learning. It does **not** assume you know anything about image segmentation. Every concept is introduced from zero, explained in plain language, connected back to this specific project, and paired with a visual wherever it helps.

NEEDS VERIFICATION: This guide does not have access to your actual dataset files. Anywhere a specific number (image size, class count, class names, split ratio) is used as an example, it is clearly marked. Confirm the real values against your dataset before writing code.

How to use this guide

- Read it start to finish once, without coding. The goal of this pass is understanding, not typing.
- The checklist and self-test near the end tell you whether you're ready to start building.
- The implementation roadmap at the very end turns this understanding into small, verifiable coding steps — one small file at a time, never a 500-line dump.

1. What This Project Is

Understanding the goal before touching any code

Your project takes a photograph as input, and produces a second image of the exact same size, where **every pixel** has been labeled with a category. That labeled output is called a **segmentation mask**.

In one sentence: **image segmentation means classifying every single pixel of an image, instead of classifying the whole image at once.**

1.1 How segmentation differs from other vision tasks

These four terms get confused constantly. They all involve looking at images, but they answer completely different questions:

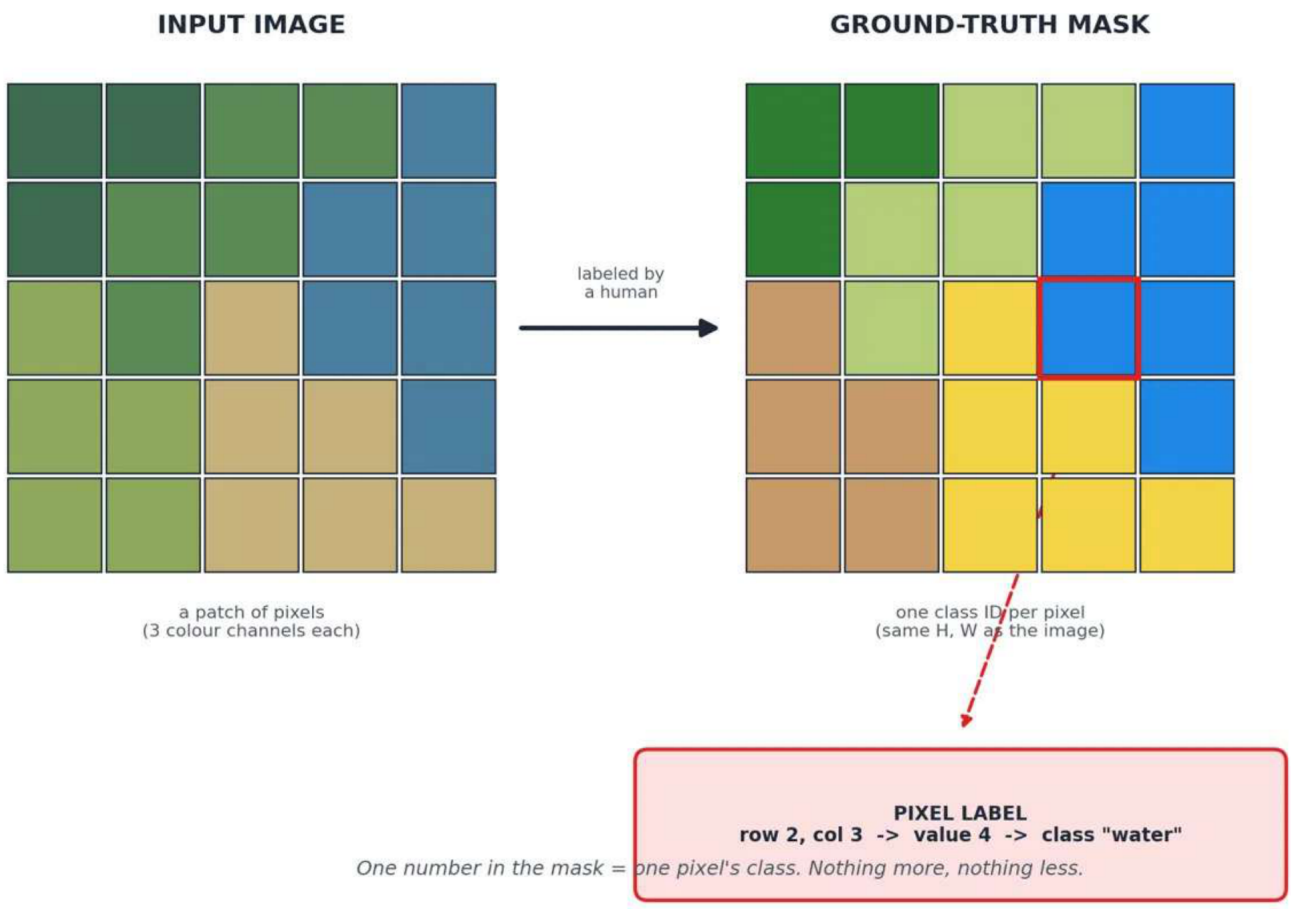
Task	Question it answers	Output shape
Image classification	“What is the main thing in this image?”	One label for the whole image
Object detection	“Where are the objects, roughly?”	Boxes (x, y, w, h) + a label per box
Semantic segmentation	“What category is EVERY pixel?”	One label per pixel (this project)
Instance segmentation	“What category, AND which specific object, is every pixel?”	One label per pixel + an object ID

This project is a **semantic** segmentation project unless stated otherwise: we care about the category of each pixel (e.g. “road”), not about telling two different cars apart individually.

NEEDS VERIFICATION: Confirm whether your project needs semantic segmentation (categories only) or instance segmentation (categories + individual object identity). The rest of this guide assumes semantic segmentation, which is the standard starting point and matches a U-Net-style architecture.

1.2 What a segmentation mask actually looks like

A mask is not a pretty picture — it's a grid of integers, exactly the same height and width as the input image. Each integer says which class that pixel belongs to. When you display a mask, you're just showing a color for each integer so a human can see it easily.



The image on the left is what a human (or camera) sees. The mask on the right stores one class ID for every single pixel — the color is just how we visualize that integer.

1.3 What one pixel in the mask represents

Take any single pixel location, say row 2, column 3. In the input image, that location holds 3 numbers (red, green, blue intensity). In the mask, that exact same location holds **one** number: the class ID. Nothing more. A mask value of, say, 4 doesn't mean “intensity 4” — it means “this pixel belongs to class number 4, whatever that class is (e.g. 'water')”.

1.4 What the model receives and produces

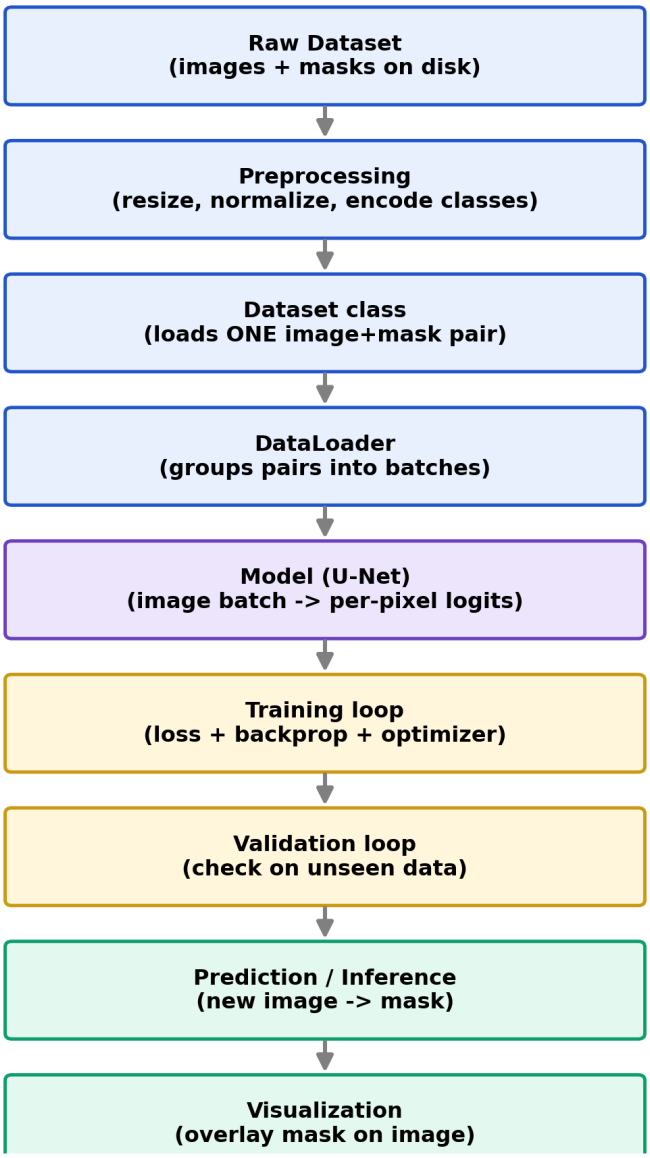
	Shape	Meaning
Model INPUT	3 × H × W	One image, 3 color channels, height H, width W
Model OUTPUT	C × H × W	For every pixel, a raw score for each of the C classes

The model does NOT directly output a mask. It outputs a **score per class per pixel** (called logits). Turning that into an actual mask is one extra step (`argmax`) covered in detail in the Tensor Shapes section.

1.5 The complete pipeline

Every segmentation project — including this one — follows the same overall shape. Everything in this guide is just a deep dive into one box of this diagram at a time:

The complete project pipeline



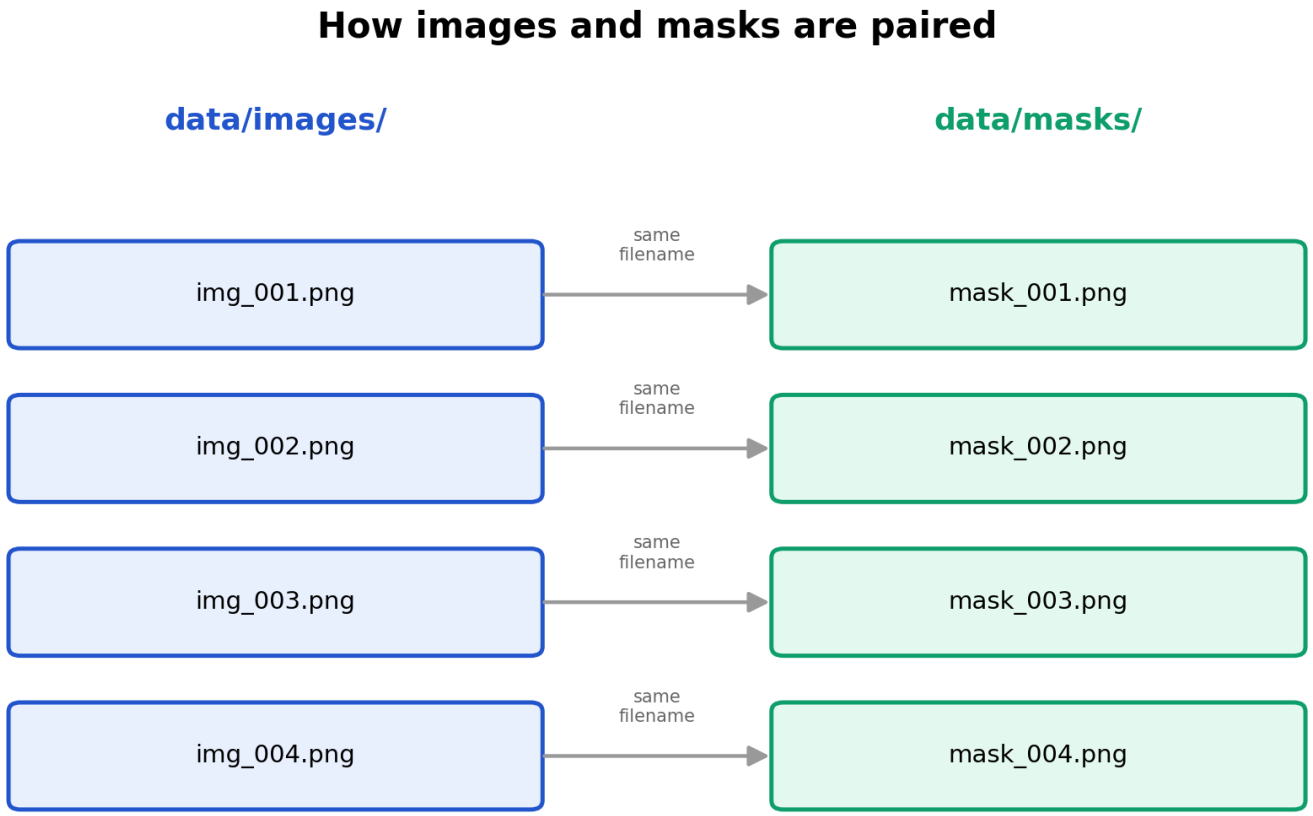
2. Dataset Overview

What you actually have on disk, in plain terms

NEEDS VERIFICATION: No dataset files or dataset documentation were available while writing this guide, so the specific numbers below (image size, class count, class names, split ratio) are illustrative placeholders, not facts about your dataset. Run the small inspection code in Stage 1 of the next section on your real data and fill these in yourself before building anything.

2.1 Images and masks, in simple terms

- **Input images** — ordinary photos/images, stored as 3-channel color (RGB), one file per sample.
- **Masks** — one file per image, same width and height as the image, but storing a class ID (a small integer) at every pixel instead of color.
- **Pairing** — each image has exactly one matching mask. They're usually paired by having the same filename in two different folders (or the same index in two arrays).



The image and its mask are paired by matching filenames (or a fixed ordering).
Each pair is ONE training example.

2.2 Key facts you need before writing any code

Fact	Why it matters	Status
Image height × width	Sets the input size your model and preprocessing must handle	needs verification
Mask height × width	Should match the image (or be resizable to match)	needs verification
Number of classes (C)	Sets the number of output channels of your model	needs verification
Class ID → class name mapping	Needed to interpret predictions and compute per-class metrics	needs verification
Binary or multi-class	Binary = 2 classes (e.g. object / background). Multi-class = 3+.	needs verification

2.3 A tiny artificial example

Before looking at real data, it helps to see a fake, tiny example small enough to read every number. Imagine a 3×3 image (so 9 pixels), with 4 classes: 0 = background, 1 = road, 2 = building, 3 = tree.

```
Mask (3x3, one class ID per pixel):

    0   0   1
    0   2   2
    3   3   2
```

Reading this: the pixel at row 0, column 2 has value 1 → it belongs to class 1 (“road”). The pixel at row 2, column 0 has value 3 → class 3 (“tree”). That's the entire idea of a mask — it's just a grid of “which class is this pixel” answers. Real masks are the same idea at, for example, 512×512 pixels instead of 3×3.

2.4 Train / validation / test data

You don't train on 100% of your data. You split the image-mask pairs into three separate groups that never overlap:

Split	Used for	Rule
Train	Actually updating the model's weights	The model sees these many times, in every epoch
Validation	Checking progress DURING training, tuning choices	The model never trains on these, but you look at the results often
Test	One final, honest check at the very end	Touched only ONCE, after all decisions are already made

Splitting pairs into train / validation / test



Example split shown for illustration only — exact ratio and counts for THIS dataset need verification.

2.5 Why the dataset needs preprocessing at all

Images straight off disk vary in size, are stored as 0–255 integers, and come in a channel order (or file format) that PyTorch doesn't expect by default. Masks also need to be turned into clean small integers (0, 1, 2, 3...) with no stray values. Preprocessing makes every sample consistent, so the model always receives

input in the exact same shape and scale. The full reasoning is in the dedicated Preprocessing section later in this guide.

3. The Project, Broken Into 22 Stages

Every stage explained as WHAT / WHY / HOW / WATCH OUT

This is the backbone of the whole guide. Every stage of the project is broken into four questions. Read this section slowly — everything you build later is just a code version of one of these boxes.

STAGE 1 — Understand the dataset

WHAT	Look at what data you actually have — how many images, how many masks, their formats, and how they're organized on disk.
WHY	Every later decision (model input size, number of classes, loss function) depends on facts about the data. Skipping this step means building blind.
HOW	Open a few image/mask pairs directly with PIL or matplotlib. Print their shapes, data types, and the unique values inside the masks.
WATCH OUT	Assuming details instead of checking them (“surely it's 256×256”). Not noticing that an image and its mask have mismatched sizes or a different file order.

STAGE 2 — Inspect images and masks

WHAT	Visually display several image/mask pairs side by side.
WHY	Bugs in segmentation projects are usually invisible in code but obvious in a picture — a misaligned mask or a blank mask jumps out instantly when you actually look at it.
HOW	matplotlib subplots: image on the left, the mask (colored per class) on the right, for 5–10 random samples.
WATCH OUT	Trusting data is correct just because it loaded without an error. A tensor can load fine and still be wrong.

STAGE 3 — Understand class labels

WHAT	Figure out exactly which integer in the mask corresponds to which real-world category (e.g. 0 = background, 1 = road, 2 = building).
WHY	Every part of the model, loss, and metrics downstream assumes one specific integer → class mapping. Get it wrong and your “accuracy” numbers mean nothing.
HOW	Find the dataset's label list (often a JSON/YAML/text file), or infer it by listing unique values across several masks and checking documentation.
WATCH OUT	Guessing the mapping. Forgetting an “ignore” / “unlabeled” class that shouldn't be scored.

STAGE 4 — Train / validation / test split

WHAT	Divide image-mask pairs into three non-overlapping groups: one to train on, one to check progress with, one held back for a final honest evaluation.
WHY	Evaluating on data the model has already seen gives a falsely optimistic picture. Validation catches overfitting early; test tells the real story at the end.
HOW	Shuffle pair indices with a fixed random seed, then slice into roughly 70/15/15 (adjust for your dataset's actual size — needs verification).
WATCH OUT	Splitting AFTER augmenting, so near-duplicate augmented copies of the same image leak across splits. Not fixing the random seed, so the split changes every run.

STAGE 5 — Preprocessing

WHAT	Transform raw images and masks into a consistent numeric format the model can consume: resizing, normalizing pixel values, converting masks to clean class IDs.
WHY	A network expects fixed-size numeric tensors on a stable scale. Raw images differ in size and sit on a 0–255 integer scale, which is unstable for training.
HOW	Resize image and mask with DIFFERENT interpolation methods (see the dedicated Preprocessing section). Scale image pixels to roughly 0–1. Keep the mask as plain integer class IDs.
WATCH OUT	Resizing masks with bilinear/bicubic interpolation, which invents fractional “blended” values that match no real class at all.

STAGE 6 — Data augmentation

WHAT	Create extra training variety on the fly with random flips, rotations, crops, and color changes.
WHY	Segmentation datasets are often small. Augmentation exposes the model to more visual variation without collecting new images, reducing overfitting.
HOW	Apply the SAME geometric transform to the image and its mask together (a library like albumentations does this automatically). Apply color-only changes to the image alone.
WATCH OUT	Transforming the image but forgetting to apply the identical transform to the mask — this silently misaligns every augmented sample.

STAGE 7 — Create a Dataset class

WHAT	Write a small Python class that knows how to load ONE (image, mask) pair given an index, and apply preprocessing/augmentation to it.
WHY	PyTorch's data-loading system needs a standard interface (__len__ and __getitem__) to know how many samples exist and how to fetch any one of them.
HOW	Subclass torch.utils.data.Dataset. Store file paths in __init__. In __getitem__, load the pair, transform it, and return two tensors.
WATCH OUT	Doing heavy work (e.g. loading the ENTIRE dataset into memory) inside __init__ instead of loading lazily per-sample. Returning mismatched image/mask shapes.

STAGE 8 — Create a DataLoader

WHAT	Wrap the Dataset in a DataLoader that automatically groups samples into batches and shuffles them each epoch.
WHY	Training one image at a time is slow and noisy; training on the WHOLE dataset at once often doesn't fit in memory. Batching is the practical middle ground.
HOW	`DataLoader(dataset, batch_size=8, shuffle=True)` for training; `shuffle=False` for validation/test.
WATCH OUT	Forgetting shuffle=True for training (the model can learn spurious order-based patterns). Choosing a batch size too large for available memory.

STAGE 9 — Understand the segmentation model architecture

WHAT	Understand, conceptually, how a network can take an image and output a full-resolution map of per-pixel predictions.
WHY	If you don't understand WHY the architecture looks the way it does, you can't debug it when it misbehaves or adapt it to a new project.
HOW	Study the encoder → bottleneck → decoder pattern with skip connections (full deep-dive in the dedicated Model Architecture section).
WATCH OUT	Copy-pasting a model class without understanding its shape flow, then being unable to fix a shape-mismatch error.

STAGE 10 — Understand encoder/decoder concepts

WHAT	The encoder shrinks the image while learning WHAT is in it; the decoder grows it back while deciding WHERE things are.
WHY	Segmentation needs both “what” (semantic understanding) and “where” (precise pixel location) — no single half of the network can do both well alone.
HOW	Trace the U-Net diagram in the Model section: follow the blue boxes (encoder) shrinking, then the green boxes (decoder) growing back.
WATCH OUT	Believing the encoder alone “is” the model — the decoder is equally essential for segmentation, unlike plain classification.

STAGE 11 — Understand input/output tensor shapes

WHAT	Track exactly what shape the data has at every step: on disk, as a tensor, as a batch, after the model, and after argmax.
WHY	The #1 source of bugs in segmentation code is a silent shape mismatch — important enough to get its own full section later in this guide.
HOW	Print `.shape` after every step during your first debugging run and compare it against what you expect.
WATCH OUT	Assuming shapes are correct just because the code “ran”. PyTorch broadcasts silently in some cases, which hides real bugs.

STAGE 12 — Choose loss function

WHAT	Pick a formula that scores how wrong a prediction is compared to the ground-truth mask, as a single number.
WHY	The loss is the ONLY signal that guides learning — backpropagation computes gradients of the loss, not of “accuracy”.
HOW	Start with `nn.CrossEntropyLoss()`, which directly compares raw logits to integer class-ID masks (see the dedicated Loss Function section).
WATCH OUT	Applying softmax before CrossEntropyLoss (it applies softmax internally — doing it twice breaks the math). Ignoring class imbalance when one class dominates the image.

STAGE 13 — Choose evaluation metrics

WHAT	Pick numbers that describe HOW GOOD the trained model actually is, in a way a human can interpret.
WHY	Loss going down doesn't always mean the model is useful — pixel accuracy and IoU describe real segmentation quality (see the dedicated Metrics section).
HOW	Track pixel accuracy, per-class IoU, and mean IoU on the validation set after every epoch.
WATCH OUT	Reporting only pixel accuracy — on an image that's 95% background, predicting “background everywhere” scores 95% while being useless.

STAGE 14 — Training loop

WHAT	The repeating cycle that actually teaches the model: feed data in, measure error, adjust the weights, repeat many times.
WHY	This loop IS the training of a neural network — everything before this step was preparation.
HOW	For each batch: forward pass → compute loss → `loss.backward()` → `optimizer.step()` → `optimizer.zero_grad()`.
WATCH OUT	Forgetting optimizer.zero_grad() (gradients accumulate across batches instead of resetting). Forgetting model.train() mode.

STAGE 15 — Validation loop

WHAT	After each epoch, run the model on the validation set WITHOUT updating any weights, to check performance on unseen data.
WHY	This is your early warning system for overfitting — if training loss keeps dropping while validation loss rises, the model is memorizing, not learning.
HOW	Wrap the pass in `torch.no_grad()` and call `model.eval()` first.
WATCH OUT	Forgetting model.eval() or torch.no_grad() — wastes memory and can give inconsistent validation numbers.

STAGE 16 — Save checkpoints

WHAT	Periodically save the model's current weights to disk, usually whenever validation performance improves.
WHY	Training can crash, get interrupted, or overfit late — without checkpoints you'd lose hours of computation with no way to recover the best version.
HOW	`torch.save(model.state_dict(), "best_model.pt")` whenever validation IoU improves.
WATCH OUT	Only saving the FINAL epoch's weights, which may be worse than an earlier epoch if the model started overfitting.

STAGE 17 — Load trained model

WHAT	Reconstruct the model architecture in code, then load the saved weights into it.
WHY	A checkpoint file only contains numbers (weights) — it has no idea what architecture those numbers belong to, so you must rebuild the exact same architecture first.
HOW	`model = UNet(...); model.load_state_dict(torch.load(path)); model.eval()`.
WATCH OUT	Loading weights into a model built with different settings (e.g. a different number of classes) — this either crashes or loads garbage silently.

STAGE 18 — Run inference on a new image

WHAT	Feed a brand-new image (never seen during training) through the trained model to get a prediction.
WHY	This is the actual "product" — everything else was building the ability to do this one useful thing.
HOW	Preprocess the new image exactly like training images were preprocessed, add a batch dimension, run it through the model in eval() + no_grad() mode.
WATCH OUT	Preprocessing the new image DIFFERENTLY than training images (e.g. forgetting to normalize) — the model will produce nonsense.

STAGE 19 — Convert model output into a segmentation mask

WHAT	Turn the model's raw output (a score for every class, at every pixel) into ONE final class ID per pixel.
WHY	The model naturally outputs "how confident am I in each class", but a mask needs a single decided answer per pixel.
HOW	`predicted_mask = torch.argmax(logits, dim=1)` picks the class with the highest score at each pixel.
WATCH OUT	Forgetting dim=1 (arg-maxing over the wrong axis) — a very common and very confusing bug.

STAGE 20 — Visualize prediction vs ground truth

WHAT	Plot the original image, the ground-truth mask, and the predicted mask side by side.
WHY	Numbers (loss, IoU) tell you HOW WRONG the model is; pictures tell you WHY — edge errors vs. missing an entire object are very different problems only a visual reveals.
HOW	matplotlib, 3 subplots per sample: image / ground truth / prediction, using the same color-per-class scheme for both masks.
WATCH OUT	Only looking at aggregate metrics and never actually looking at predicted masks — this hides systematic failure patterns.

STAGE 21 — Evaluate whether the model is actually good

WHAT	Step back and honestly judge whether the model is good enough for its intended use, using both metrics and visual inspection on the held-out TEST set.
WHY	A model can have a good-looking loss curve and still fail in specific, important ways (e.g. always missing small objects) — real evaluation goes beyond one number.
HOW	Compute mean IoU on the test set; look at the worst-performing samples individually to understand failure patterns.
WATCH OUT	Reusing the validation set as if it were a test set — it isn't fully “unseen” anymore, since you used it to make decisions during training.

STAGE 22 — Final project structure

WHAT	Organize the finished code into small, single-purpose files that are easy to navigate, test, and explain.
WHY	A project you can't navigate is a project you can't debug, extend, or explain — which defeats the entire purpose of building it to learn.
HOW	Follow the recommended folder structure later in this guide — one file per responsibility, small functions, explicit code.
WATCH OUT	Letting one file (often train.py) grow to hold data loading, model definition, training, AND visualization all at once.

4. Tensor Shapes, Deeply Explained

The single most common source of bugs — worth mastering on purpose

If you only deeply understand one section of this guide, make it this one. Almost every segmentation bug traces back to a tensor having the wrong shape at some point in the pipeline.

4.1 The letters you'll see everywhere

Letter	Means	In this project
B	Batch size — how many images are processed together	e.g. 8 or 16 images per step
C	Number of classes	however many categories your masks contain (needs verification)
H	Height, in pixels	after resizing, e.g. 256
W	Width, in pixels	after resizing, e.g. 256

4.2 Why channels "move"

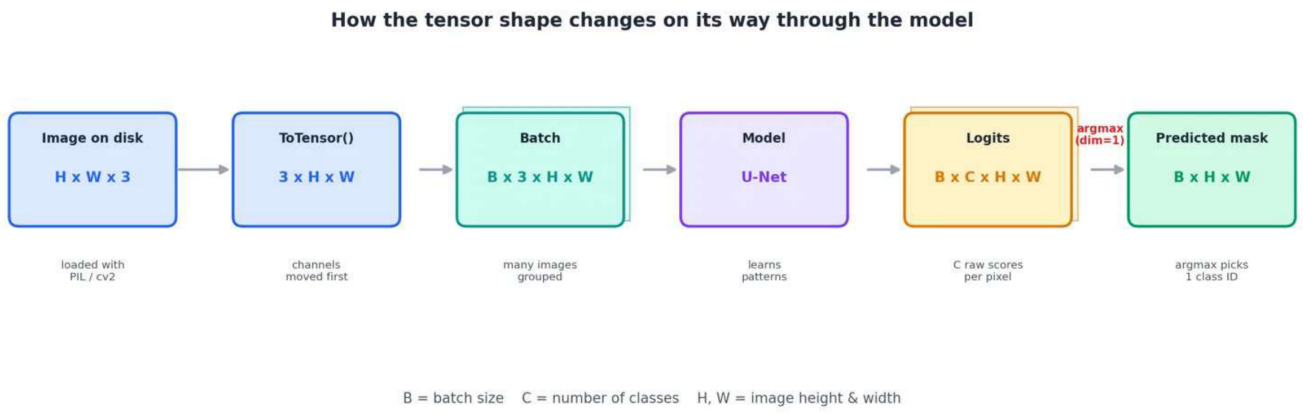
An image loaded from disk (PIL/OpenCV) is usually shaped `H × W × 3` — for every pixel position, 3 color numbers sit next to each other. PyTorch convolutions expect the OPPOSITE layout: `3 × H × W` — channels first. This isn't arbitrary: PyTorch's convolution operations are written to slide a filter across the last two dimensions (height, width) while treating the first dimension as separate channels to combine. Converting an image with `ToTensor()` (or `.permute(2,0,1)`) performs exactly this reordering.

4.3 Why the model output has C channels but the mask does not

This trips up almost everyone at first, so slow down here.

- The **mask** (ground truth) only needs to store ONE number per pixel: the correct class ID. Shape: `B × H × W`.
- The model's **output** (logits) needs to store the model's confidence for EVERY possible class, at every pixel, because the model hasn't decided yet — it's reporting raw scores. Shape: `B × C × H × W`.
- `argmax(dim=1)` is the step that collapses the C score-channels down to a single winning class ID per pixel — turning `B × C × H × W` logits into a `B × H × W` predicted mask, the same shape as the ground truth, so the two can finally be compared pixel-by-pixel.

4.4 The full transformation, step by step



4.5 Self-check

Given `B=4`, `C=6`, `H=128`, `W=128`: what is the shape of the logits, and what is the shape of the predicted mask after argmax? *(Answer: logits = 4×6×128×128, predicted mask = 4×128×128.)*

5. The Model Architecture, Explained From Zero

Not “we use U-Net” — WHAT it actually does and WHY

A plain image classifier only needs to answer “what is this?”, so it's allowed to throw away spatial detail as it goes — by the end it just needs one label. A segmentation model has a much harder job: it must answer “what is this AND exactly where?” for every pixel. That extra requirement — keeping track of WHERE things are, not just WHAT they are — is the entire reason segmentation architectures look different from classifiers.

5.1 Building blocks, one at a time

"Convolution"	
Technical:	A small filter (e.g. 3×3) slides across the image, computing a weighted sum at each position.
Simple:	A pattern detector that scans the whole image looking for one specific visual pattern (an edge, a texture, a color transition).
Example:	A vertical-edge filter lights up wherever a vertical edge exists in the image.
In this project:	Stacks of convolutions are what let the encoder recognize roads, buildings, trees, etc. in your images.

"Feature map"	
Technical:	The OUTPUT of a convolution layer — a grid of numbers showing how strongly a pattern was detected at each location.
Simple:	Think of it as a new, transformed “image” where each value says “how much of my pattern is here?” instead of storing color.
Example:	A feature map for a “vertical edge” filter is bright wherever the original image has a vertical edge.
In this project:	Each encoder block produces many feature maps (dozens to hundreds), each specialized in a different pattern.

"Downsampling"	
Technical:	Shrinking a feature map's height and width (e.g. via strided convolution or max-pooling), usually by half each time.
Simple:	Zooming out. Each step trades pixel-level detail for a bigger-picture understanding of what's in the image.
Example:	A 256×256 feature map becomes 128×128, then 64×64, then 32×32.
In this project:	This is how the encoder builds understanding of large objects and overall scene content.

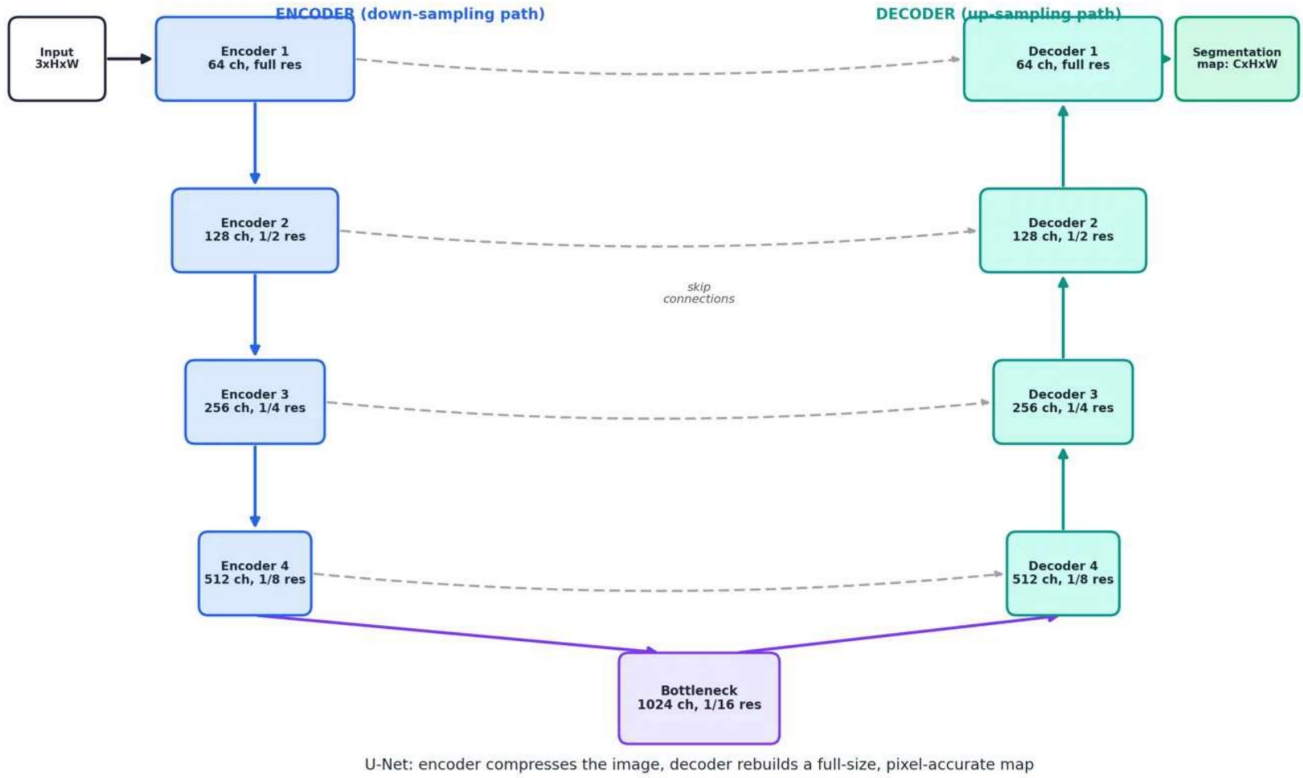
"Upsampling"	
Technical:	Growing a feature map's height and width back up (e.g. via transposed convolution or interpolation).
Simple:	Zooming back in. Reverses downsampling so the output can eventually match the original image's resolution, pixel for pixel.
Example:	A 32×32 feature map is grown back to 64×64, then 128×128, then 256×256.
In this project:	The decoder must upsample all the way back to your image's H×W so every pixel gets a prediction.

5.2 Encoder, bottleneck, decoder

Part	What it does	Analogy
Encoder	Repeatedly convolves + downsamples, building deeper “what is this” understanding at lower resolution	Squinting at the whole picture to understand its content
Bottleneck	The smallest, most compressed representation — maximum understanding, minimum spatial detail	The single most condensed summary of the image
Decoder	Repeatedly upsamples + convolves, rebuilding full resolution while placing that understanding back in space	Redrawing the picture at full size, guided by that understanding

5.3 Skip connections — why the decoder alone isn't enough

By the time the encoder reaches the bottleneck, a lot of exact pixel-position detail has been lost — that's the whole point of downsampling. But segmentation needs that detail back for sharp, accurate mask edges. **Skip connections** solve this by directly copying feature maps from each encoder stage across to the matching decoder stage, so the decoder gets both the deep understanding (from below) AND the fine spatial detail (from the skip) at every resolution level.



This is a U-Net: the encoder (blue, left) compresses the image while learning what's in it. The decoder (green, right) rebuilds full resolution. The dashed grey arrows are skip connections, carrying fine spatial detail straight across.

IMPORTANT: Why segmentation needs spatial information, restated simply: a classifier can afford to forget WHERE a cat's ear was, because it only needs to say “cat”. A segmentation model must say “cat” at exactly the pixels where the cat is — lose that spatial information and your mask edges go blurry or misplaced.

6. Loss Function, From Zero

How the model knows it's wrong

6.1 What is a loss, and why do we need one?

A loss is a single number that measures how wrong a prediction is. Low loss = good prediction, high loss = bad prediction. We need it because training a neural network means repeatedly nudging its weights in the direction that makes this number smaller. Without a loss, there's no signal telling the model which direction "better" even is.

6.2 What exactly is being compared

At every pixel, the loss compares the model's raw class scores (logits) against the single correct class ID from the ground-truth mask, then averages that comparison across all pixels and all images in the batch.

6.3 What logits are

Logits are the model's raw, un-normalized output scores for each class — before they've been converted into probabilities. A logit of `2.1` for "road" and `-0.3` for "building" simply means the model currently favors "road" more; the numbers themselves aren't probabilities yet.

6.4 Cross entropy for multi-class segmentation

Cross entropy loss internally converts logits into probabilities (via softmax) and then penalizes the model in proportion to how much probability it assigned to the WRONG classes. It does this independently at every pixel, then averages.

IMPORTANT: Never apply softmax yourself before `nn.CrossEntropyLoss()` in PyTorch. This loss function applies softmax internally as part of its calculation. Applying it twice distorts the math and produces a broken, misleadingly small gradient signal — the model appears to train, but poorly and inconsistently.

6.5 A tiny numeric example

Imagine ONE pixel whose true class is "road" (class 1 of 3: background=0, road=1, tree=2). The model outputs logits `[0.2, 2.5, 0.1]` for these classes.

```
Step 1 - softmax turns logits into probabilities (they sum to 1):
    softmax([0.2, 2.5, 0.1]) = approx [0.09, 0.86, 0.05]

Step 2 - cross entropy loss looks ONLY at the probability given
```

```
to the TRUE class (road = index 1), and takes -log() of it:
    loss = -log(0.86) = approx 0.15   (small loss - good prediction)

If the model had instead been confident in the WRONG class, e.g.
assigned road a probability of only 0.05, the loss would be:
    loss = -log(0.05) = approx 3.00   (large loss - bad prediction)
```

This is computed at every pixel and averaged into one loss value for the whole batch.

6.6 Class imbalance

In many segmentation datasets, one class (often “background”) covers far more pixels than the others. Plain cross entropy treats every pixel equally, so the model can get a deceptively low loss just by favoring the dominant class and largely ignoring rare ones.

6.7 When Dice loss / IoU-based losses help

Dice loss (and IoU-based losses) measure overlap between the predicted and true regions directly, rather than scoring each pixel independently. This makes them naturally more robust to class imbalance, since a class that's rare but geometrically compact still contributes meaningfully to the score. A common, effective combination is `CrossEntropyLoss + DiceLoss` added together.

7. Evaluation Metrics

How good is the model, really?

7.1 Pixel accuracy

The simplest metric: what fraction of all pixels did the model classify correctly? Easy to compute, but easy to game — a model that always predicts the dominant class can score high accuracy while being useless for the classes that actually matter.

7.2 IoU — Intersection over Union

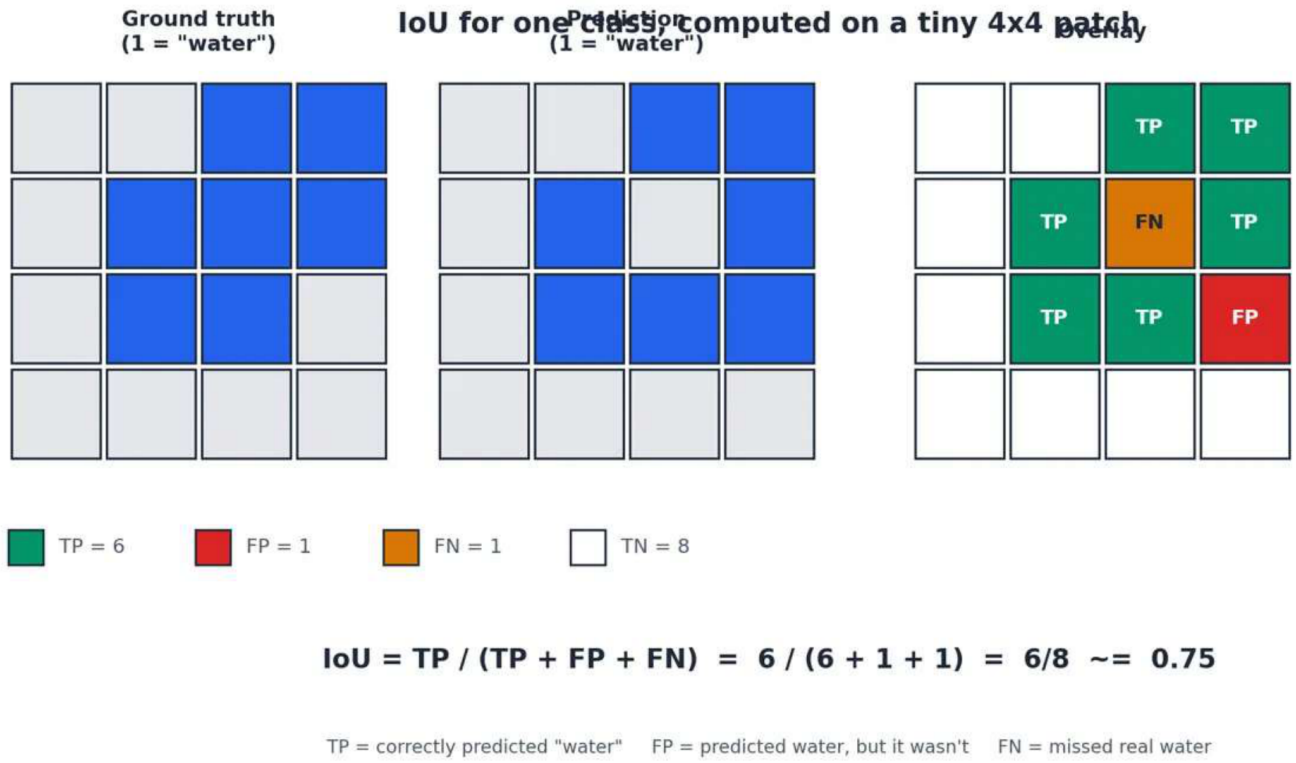
For one class, IoU measures how much the predicted region and the true region overlap, relative to their combined area:

```
IoU = (predicted region) AND (true region)
      ------------------------------------
      (predicted region) OR  (true region)
```

An IoU of 1.0 means a perfect match. An IoU of 0 means no overlap at all. This is computed **per class**, then usually averaged into a single **mean IoU (mIoU)** across all classes.

7.3 TP / FP / FN, visually

IoU is built from three counts: **True Positives** (correctly predicted that class), **False Positives** (predicted that class, but it wasn't), and **False Negatives** (missed a pixel that really was that class).



A worked example on a tiny 4×4 patch for the class “water”. IoU = TP / (TP + FP + FN) = 6 / (6+1+1) = 0.75.

7.4 Dice score

Dice score is closely related to IoU and often used alongside it:

```
Dice = 2 x TP
       -----------------
       (2 x TP) + FP + FN
```

Dice and IoU always move together (Dice is always ≥ IoU for the same prediction) — you generally only need to track one of them, but you'll see both used in papers and libraries, so it helps to recognize both.

7.5 Mean IoU

Mean IoU (mIoU) is simply the IoU for each class, averaged across all classes. It's the single most common headline number reported for segmentation models, because it treats every class fairly regardless of how many pixels it covers.

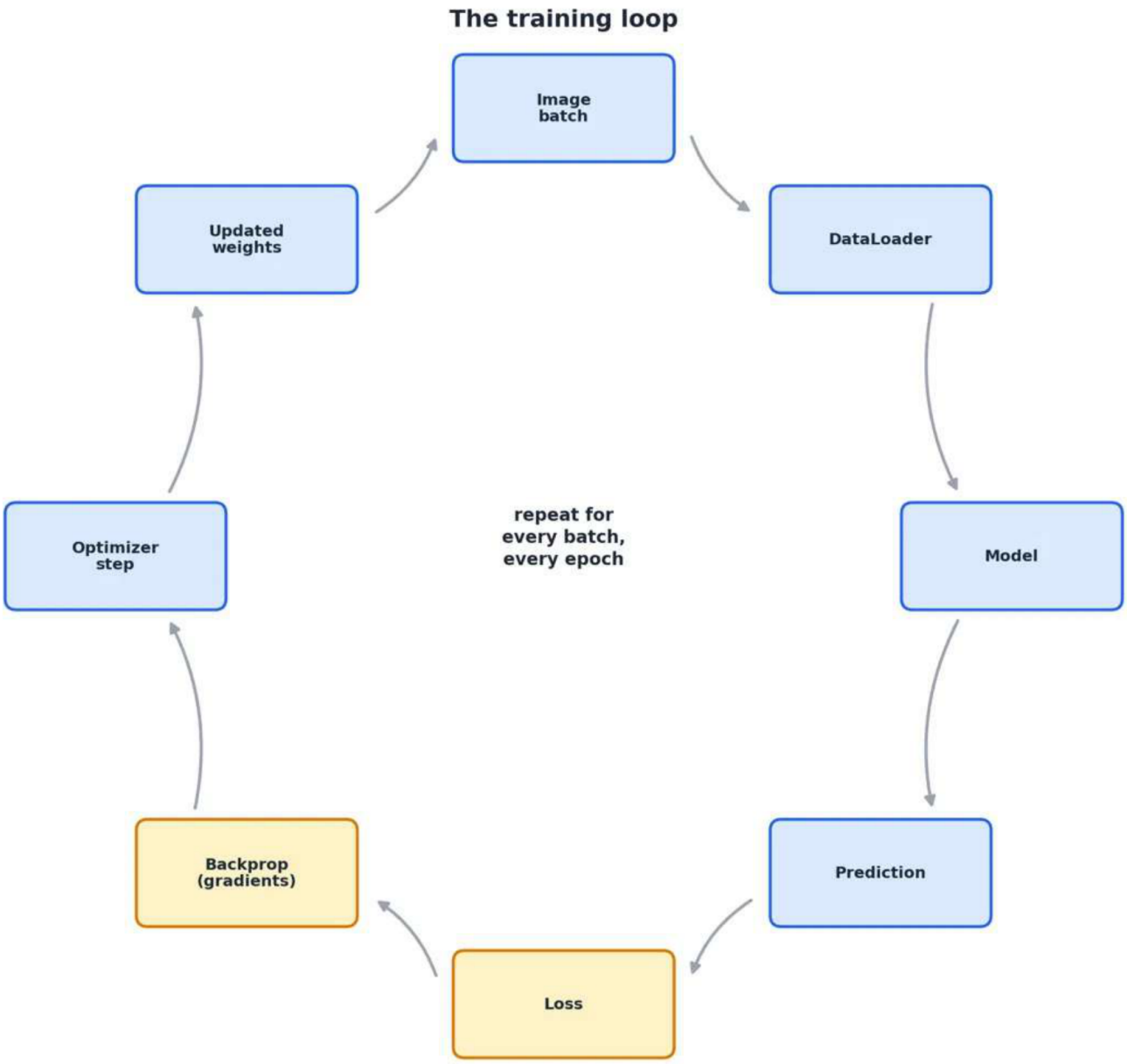
8. The Training Loop

The cycle that actually teaches the model

8.1 Vocabulary

Term	Meaning
Epoch	One full pass through the entire training dataset
Batch	A small group of samples processed together (set by the DataLoader)
Forward pass	Running input through the model to get a prediction
Backward pass	Computing how much each weight contributed to the error (gradients)
Gradient	The direction and size of the nudge that would reduce the loss for one weight
Optimizer	The algorithm that actually applies those nudges to the weights (e.g. Adam)
Learning rate	How big each nudge is — too large and training is unstable, too small and it's painfully slow
Validation	Checking performance on unseen data, without updating weights

8.2 The loop, step by step



8.3 In plain English

- Grab one batch of images and their masks from the DataLoader.
- Run the images through the model — this is the **forward pass**, producing logits.
- Compare the logits to the true masks using the loss function — this produces one loss number for the batch.
- Call `loss.backward()` — PyTorch works backwards through the model computing the **gradient** for every weight (how much it should change, and in which direction, to reduce the loss).

- Call `optimizer.step()` — every weight is nudged slightly, based on its gradient and the learning rate.
- Call `optimizer.zero_grad()` to clear gradients before the next batch (they accumulate otherwise).
- Repeat for every batch in the dataset — that's one epoch. Repeat for many epochs.
- After each epoch, run a validation pass (no weight updates) to check whether the model is actually improving on unseen data.

IMPORTANT: After reading this section, you should be able to explain the training loop out loud to another student, in your own words, without looking at this page. If you can't yet, re-read section 8.3 slowly with the diagram in front of you.

9. Preprocessing, Carefully

Getting raw files ready for the model

9.1 Resizing

Both the image and the mask need to end up at the same fixed height and width so they can be batched together. But — and this is the single most important detail in this section — **they must be resized differently.**

9.2 Normalization

Image pixel values (0–255) are rescaled to a smaller, more stable range (commonly 0–1, or standardized using a mean/std per channel). This helps training converge faster and more stably, since neural networks generally train better when inputs are on a small, centered scale. Masks are **not** normalized — they stay as exact integer class IDs.

9.3 Converting image to tensor

Loaded image → PyTorch tensor: reorders `H × W × 3` into `3 × H × W` and converts to floating point (see the Tensor Shapes section for why channels move).

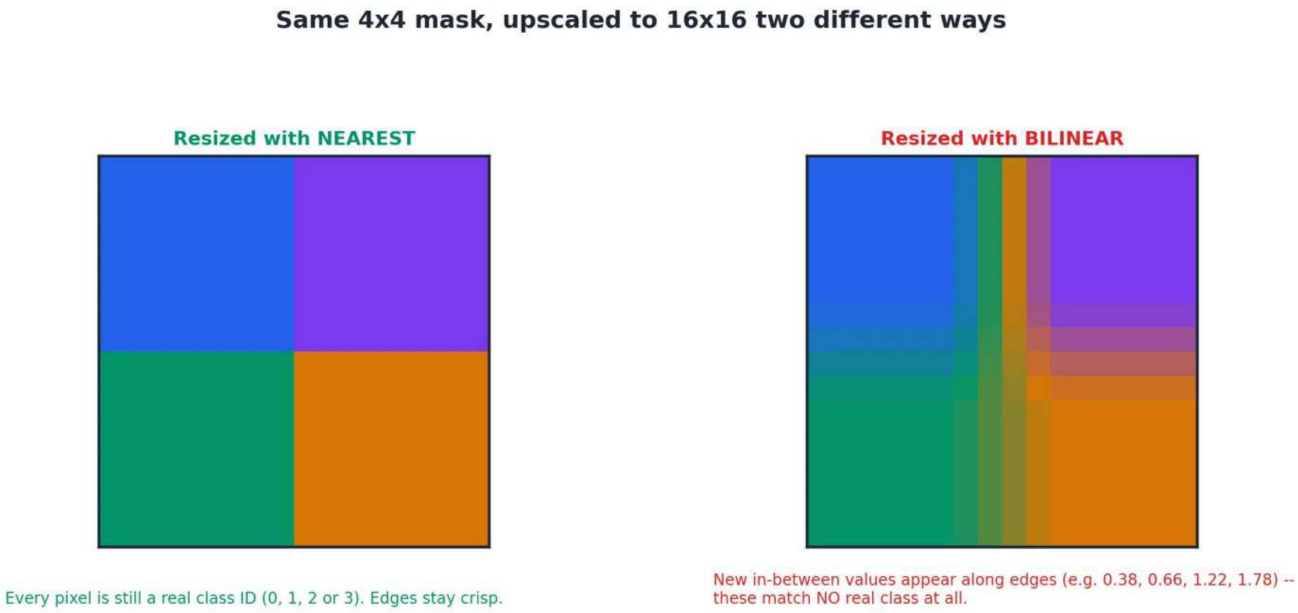
9.4 Converting mask to class IDs

Some datasets store masks as colors (e.g. RGB) rather than plain integers. If so, this step maps each unique color to a single integer class ID, using a lookup table. If your masks are already single-channel integer images, this step is just loading them without extra conversion.

9.5 Why image and mask interpolation MUST differ

When an image is resized, blending nearby pixel colors (bilinear/bicubic interpolation) looks smooth and natural — colors are continuous, so a blend of two colors is still a perfectly valid color.

A mask is completely different: its values are category labels, not continuous quantities. Blending class ID 1 (road) with class ID 2 (building) does NOT produce a meaningful in-between class — it produces a nonsense fractional value that corresponds to nothing.



Same 4×4 mask, upscaled two ways. NEAREST keeps every value a valid class ID. BILINEAR invents blended edge values that match no real class — silently corrupting your labels.

IMPORTANT: Always resize masks with NEAREST-neighbor interpolation (e.g. `Image.NEAREST` or `cv2.INTER_NEAREST`). Only images get bilinear/bicubic.

10. Data Augmentation, Synchronized

More variety, without breaking alignment

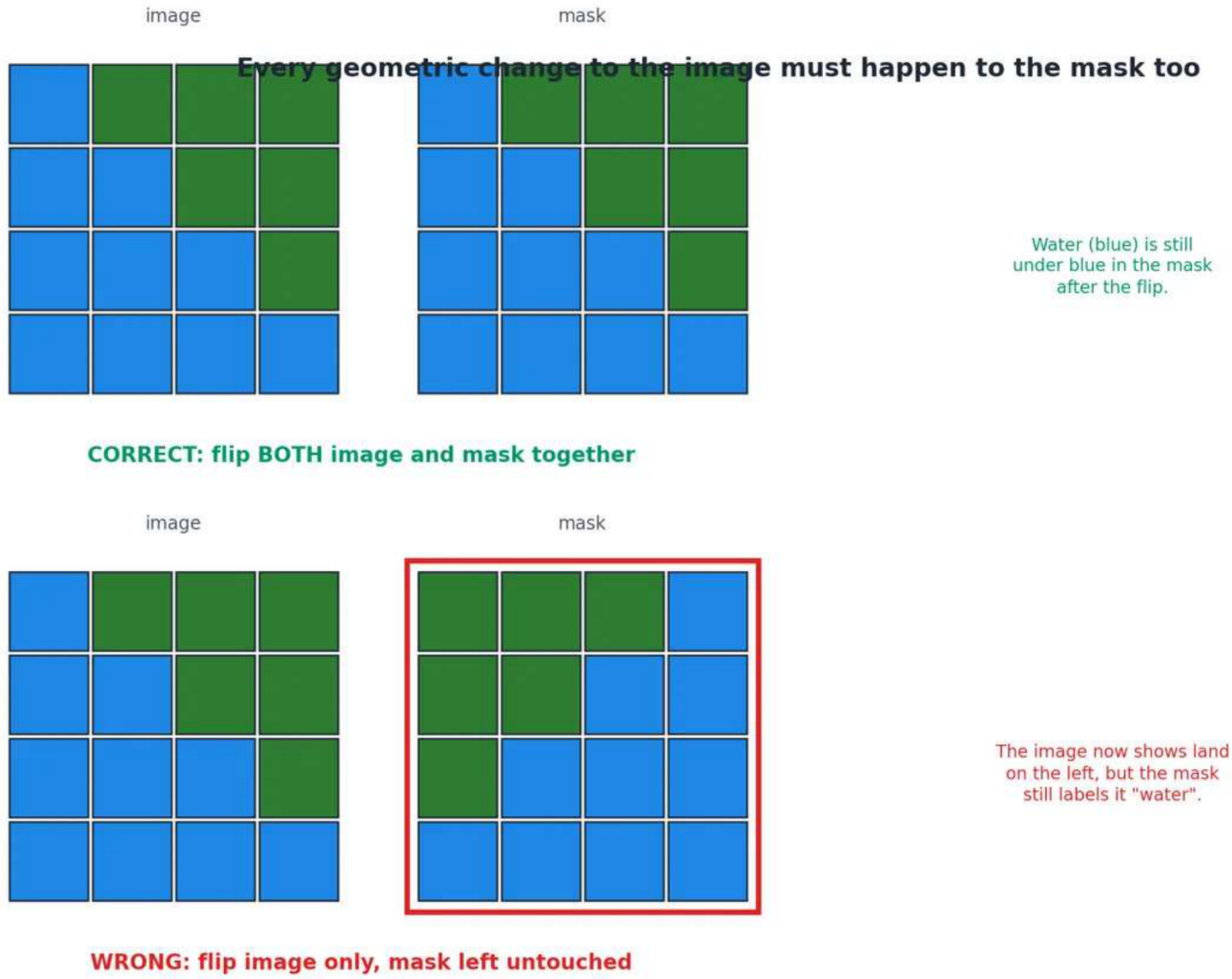
10.1 Common augmentations

Augmentation	What it does
Flipping	Mirrors the image horizontally and/or vertically
Rotation	Rotates the image by some angle
Cropping	Cuts out a smaller region of the image
Scaling	Zooms the image in or out
Color augmentation	Randomly adjusts brightness, contrast, saturation, hue

10.2 Geometric vs. color augmentation

This distinction matters a lot: **geometric** transforms (flip, rotate, crop, scale) change WHERE pixels are, so they must be applied identically to both the image and the mask. **Color** transforms (brightness, contrast, hue) only change pixel VALUES in the image and have no meaningful effect on a mask — they're applied to the image only.

10.3 What goes wrong if only the image is transformed



Top: flip both image and mask together — the labels stay correctly aligned to the objects. Bottom: flip only the image — the mask now labels the wrong pixels entirely.

IMPORTANT: If you use a library like albumentations, pass the image and mask into the SAME transform call together — it keeps geometric operations synchronized for you automatically. If you write transforms by hand, you must apply each geometric operation to image and mask with the exact same parameters (same crop coordinates, same rotation angle, etc.).

11. Project Folder Structure

Small, single-purpose files — on purpose

Every file below has exactly ONE job. If you ever find yourself unsure which file a piece of code belongs in, that's a sign the responsibility isn't clear yet — stop and decide before writing it.

Recommended project folder structure

```
project/
|
|-- data/
|   |-- images/            (raw input images)
|   |-- masks/              (ground-truth masks)
|
|-- src/
|   |-- dataset.py          Dataset class: loads ONE (image, mask) pair
|   |-- preprocessing.py    resize / normalize / encode masks
|   |-- augmentation.py     synchronized image+mask transforms
|   |-- model.py            the U-Net architecture (small, readable)
|   |-- losses.py           CrossEntropy / Dice loss definitions
|   |-- metrics.py          IoU / pixel accuracy / Dice score
|   |-- train.py            the training loop
|   |-- predict.py          run inference on a new image
|   |-- visualize.py        plot image / mask / prediction
|
|-- checkpoints/            saved model weights (.pt files)
|-- outputs/                predicted masks, saved plots
|-- notebooks/               scratch exploration (not production code)
|-- config.py                 all hyperparameters in ONE place
|-- README.md
```

What each file is responsible for

File	One-sentence responsibility
dataset.py	Loads ONE (image, mask) pair for a given index.
preprocessing.py	Resize / normalize / encode masks into class IDs.

augmentation.py	Synchronized random transforms for image + mask.
model.py	Defines the U-Net architecture — nothing else.
losses.py	Defines the loss function(s) used for training.
metrics.py	Computes IoU / pixel accuracy / Dice score.
train.py	Runs the training + validation loop.
predict.py	Runs inference on a new image.
visualize.py	Plots image / mask / prediction for inspection.
config.py	All hyperparameters and paths, in one place.

12. Implementation Philosophy

Read this before you write any code

You are building this project to **learn**, not merely to obtain working code. That single sentence should shape every decision that follows.

Do NOT:

- Optimize for the minimum number of lines.
- Create unnecessarily complicated abstractions.
- Generate huge scripts (the 500–700 line files that caused this restart).
- Hide important logic inside helper functions you can't explain yourself.

Prefer:

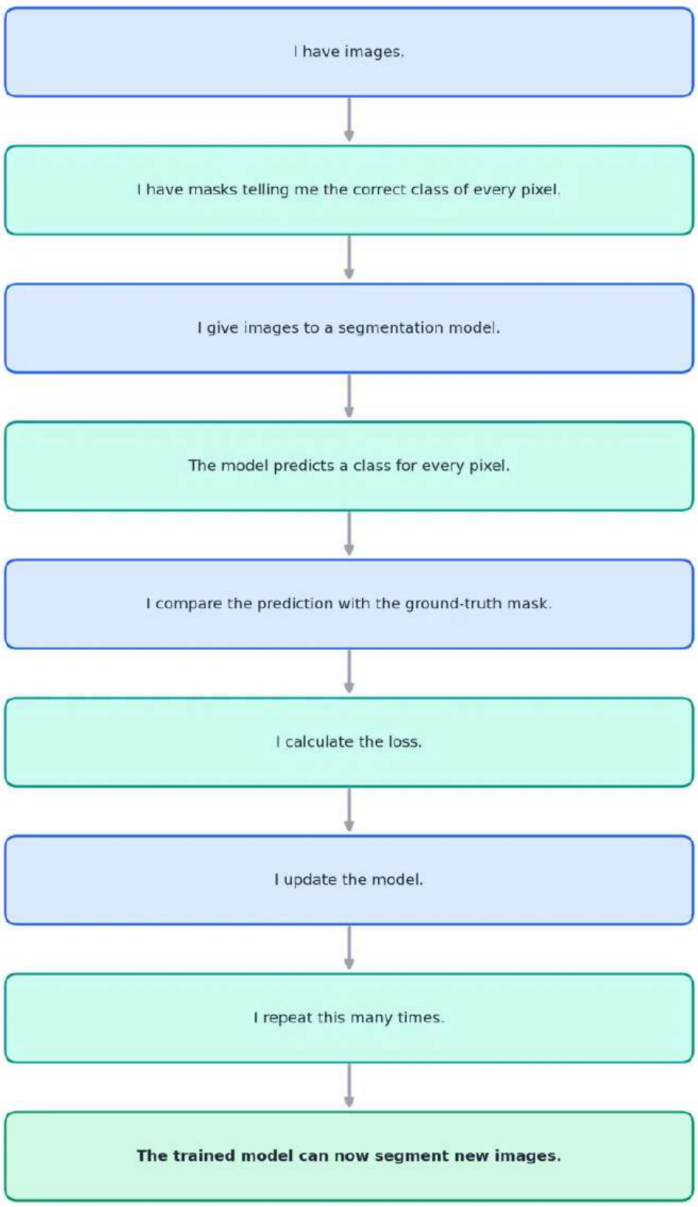
- Small files, each with one clear job.
- Small functions you can read top-to-bottom in one glance.
- Explicit code over “clever” code.
- Meaningful variable names (`image_tensor`, not `x1`).
- Comments explaining WHY a line exists, not just what it does.
- A simple architecture you fully understand over a fancier one you don't.
- Code that's easy to debug and step through.

IMPORTANT: Every implementation step, when you get to coding, should be small enough that you can fully understand it before moving to the next one. If a step feels too big to explain back to yourself, split it into two smaller steps.

13. The Mental Model

The entire project, in one breath

If you forget every diagram in this guide except one, make it this one. Everything you build maps onto one of these steps.



14. Before You Code: Checklist & Self-Test

A honest gate before you open an editor

14.1 Checklist

Go through this list slowly. If any box feels shaky, re-read that section before continuing.

- ■ I understand what a segmentation mask is
- ■ I understand image dimensions
- ■ I understand mask dimensions
- ■ I understand class IDs
- ■ I understand the train / validation / test split
- ■ I understand preprocessing, and why masks resize differently than images
- ■ I understand augmentation, and why image + mask transforms must stay synchronized
- ■ I understand what a Dataset class is responsible for
- ■ I understand what a DataLoader is responsible for
- ■ I understand the model's input and output shapes
- ■ I understand what logits are
- ■ I understand what CrossEntropyLoss compares, and why not to apply softmax before it
- ■ I understand IoU
- ■ I understand the training loop, step by step
- ■ I understand inference (running the model on a new image)

14.2 Self-test

Try answering these from memory before checking the italicized answers.

Q. What is the key difference between semantic segmentation and instance segmentation?

A. Semantic segmentation labels pixels by category only; instance segmentation also distinguishes between separate objects of the same category.

Q. Why does a segmentation model's output have C channels, while the ground-truth mask does not?

A. The model reports a raw score for every possible class at each pixel (it hasn't decided yet); the mask only needs to store the single correct class ID.

Q. What operation turns model logits into a final predicted mask, and along which dimension?

A. argmax, along dim=1 (the class/channel dimension).

Q. Why must masks be resized with nearest-neighbor interpolation instead of bilinear?

A. Bilinear blends neighboring values into fractional numbers that match no real class ID; nearest-neighbor preserves valid class IDs exactly.

Q. If you flip an image during augmentation but forget to flip its mask, what happens?

A. The mask no longer lines up with the objects in the image — labels become misaligned with what they're supposed to describe.

Q. Why should you never apply softmax before nn.CrossEntropyLoss in PyTorch?

A. CrossEntropyLoss applies softmax internally; applying it twice distorts the loss and produces a broken training signal.

Q. What does IoU measure, in one sentence?

A. The overlap between the predicted region and the true region, relative to their combined (union) area.

Q. Why is pixel accuracy alone a misleading metric for segmentation?

A. A model that just predicts the dominant class (e.g. background) can score high accuracy while being useless for the classes that matter.

Q. What is the purpose of the validation set during training?

A. To check the model's performance on unseen data after each epoch, catching overfitting before it goes too far — without ever being trained on.

Q. Why is the test set only touched once, at the very end?

A. Because using it earlier (to tune decisions) would make it no longer a fully honest, unseen measure of final performance.

15. Implementation Roadmap

Turning understanding into small, verifiable steps

This is the practical plan for when you're ready to start coding — in a NEW conversation, or once you explicitly ask for code. No phase here produces a giant file; each one produces something small enough to fully understand and verify before moving on.

PHASE 1 — Understand and inspect the dataset

File	None yet — a scratch script/notebook
Code	Load a few images + masks, print their shapes/dtypes, list the unique values found inside the masks
Output	Printed shapes, dtypes, and a couple of correctly-aligned image/mask plots
Verify	You can state, in one sentence, the image size, mask size, and number of classes
Concept first	What a mask is: one class ID per pixel

PHASE 2 — Write the smallest possible dataset loader

File	src/dataset.py
Code	A Dataset class with __len__ and __getitem__ returning (image_tensor, mask_tensor) for one index — no augmentation yet
Output	dataset[0] returns two tensors with the expected shapes
Verify	Printed shapes match your expected H×W×3 image and H×W mask (or your chosen tensor convention)
Concept first	The Dataset vs. DataLoader responsibility split

PHASE 3 — Visualize one image + mask

File	src/visualize.py
Code	A function that takes an (image, mask) pair and plots them side by side
Output	A correctly-aligned image/mask plot for 3–5 samples pulled from your Dataset class
Verify	The mask visually lines up with real objects in the image
Concept first	How to sanity-check data visually, not just by printed shape

PHASE 4 — Build the model

File	src/model.py
Code	A small U-Net: encoder, bottleneck, decoder, skip connections, final 1×1 conv to C channels
Output	model(dummy_input).shape equals (B, C, H, W)
Verify	You can explain, out loud, what each block of the model does
Concept first	Encoder / decoder + skip connections

PHASE 5 — Train on a tiny subset

File	src/train.py, config.py
Code	The training loop, run on just 5–10 images, no augmentation, a few epochs
Output	Loss decreasing sharply (the model should be able to near-memorize 5–10 images)
Verify	If loss doesn't drop on a tiny subset, something is broken BEFORE you scale up
Concept first	Forward pass, loss, backward pass, optimizer step

PHASE 6 — Verify predictions

File	src/predict.py
Code	Run the tiny-subset-trained model on those same images, argmax the output, visualize
Output	Predicted masks that roughly resemble the ground truth (the model memorized them)
Verify	Visually compare prediction vs. ground truth using your Phase 3 visualization function
Concept first	Logits → argmax → predicted mask

PHASE 7 — Train properly

File	src/train.py, src/augmentation.py, src/losses.py, src/metrics.py
Code	Full training loop on the full train/val split, with augmentation, checkpointing, and metric tracking
Output	Training/validation loss and IoU curves over many epochs
Verify	Validation IoU improves over epochs and checkpoints are being saved
Concept first	Overfitting, validation, checkpointing

PHASE 8 — Evaluate

File	A small evaluation script (or extend predict.py)
Code	Load the best checkpoint, run on the held-out TEST set, compute mean IoU / Dice / pixel accuracy
Output	Final test-set metrics + a handful of best/worst visualized predictions
Verify	You can state one specific weakness of the model (e.g. “struggles with small objects near edges”)
Concept first	Test set = final honest evaluation, used only once

PHASE 9 — Inference / demo

File	src/predict.py (finalized)
Code	A clean function: given any new image path, preprocess → model → argmax → save/plot mask
Output	A working “give it any image, get a mask back” demo
Verify	It works correctly on an image NOT from the original dataset at all
Concept first	The full pipeline, end to end, in your own words

IMPORTANT: Do not start writing actual project code from this guide alone in this same sitting. Re-read the checklist in section 14 first, and when you're ready, ask explicitly for Phase 1 — one small, verifiable step at a time.